One agent, three builds: where does the user's identity live?

Rule tested: user A may query any season; user B only 2025 onward. Enforced at the Gateway by a Cedar policy, not by the model.

1 — Runtime: your agent code holds the token

you write the loop

User
Signs in (Cognito)

→ JWT

Runtime (your Strands code)
JWT authorizer; `Authorization` header allow-listed; code forwards it

→ user's JWT

Gateway (JWT) + Cedar
Validates the token, decides per user

→ if allowed

Lambda
Gets `user_id` bound from the token

2 — Harness with a managed Gateway tool (IAM): the Gateway sees the harness, not the person

no code

User / caller
`--user-id` is only a label

→ IAM

Harness (managed loop)
Model + tools by config; runs inside a Runtime

→ harness role

Gateway (IAM) + Cedar
Sees one service role; cannot tell A from B

→

Lambda
Same metadata for A and B

3 — Harness with inline tools + your client app: the app holds the token

harness loop, your app runs tools

User
Signs in (Cognito)

→ JWT

Client app (outside Bedrock)
Holds the user's token. Calls the harness with its own IAM credentials. Sets `user_id` from the token.

↔ tool_use / toolResult

Harness (managed loop)
Picks a tool, then **stops** and hands the call back. Keeps the conversation.

App runs the tool
Sends the call to the Gateway with `Authorization: Bearer <user token>`

→ user's JWT

Gateway (JWT) + Cedar
Decides per user

→ if allowed

Lambda
Gets `user_id` from the token

Observed (same question, same model)

Test	1 Runtime	2 Harness + IAM gateway	3 Harness + client app
B asks for 2023	**denied**	**answered in full**	**denied**
B asks for 2025	**allowed**	**allowed**	**allowed**
Lambda sees the caller	user id	no (A = B)	user id
Agent code written	`main.py` + deps	none	app code, no agent packaging
Console playground	n/a	works	stops at the tool call

Takeaways

- The Gateway decides; **your code makes the decision apply to the right person**.
- I found no harness option to forward the user's token (absence in what I read and tried, not proof).
- The harness kept the conversation; the app sent only the new message on the same session.
- A model refusing an injection is luck. The control is a tool with no user-id parameter.
- A denial is HTTP 200 with an `error` in the body.

Comparison page: AWS docs, "AgentCore harness vs. Runtime". Learning prototype in a personal AWS account, built while learning with Claude (Claude desktop app). No client or proprietary data. Observed October 2026; the working rule is the author's reading of the evidence, not an AWS statement.